Exception handling

CSC103 Programming Fundamentals / Lecture 24

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Define validateMark(int marks) that throws IllegalArgumentException outside 0..100. Test -1, 0, 100 and 101.

Write a validator that throws when marks are outside 0..100.

Learning objectives

- Explain propagation and the exception hierarchy
- Use try, throw and catch
- Distinguish checked and unchecked exceptions

CLO-3
Exceptional control flow
Exception hierarchy
Checked and unchecked exceptions
Throwing and catching

Exceptional control flow

- An exception interrupts the normal flow at the failure point.
- Java searches for an applicable handler.
- Statements after the failure in the same try body are skipped.

Example

```
try {
  int n = Integer.parseInt("cat");
  System.out.println(n);
} catch (NumberFormatException ex) {
  System.out.println("Invalid integer");
}
```

Exception hierarchy

- Throwable is the common base for Error and Exception.
- RuntimeException is a subclass of Exception.
- Most application recovery targets specific Exception subclasses.

Example

```
Throwable
  Error
  Exception
    RuntimeException
```

Checked and unchecked exceptions

- Checked exceptions must be caught or declared with throws.
- Unchecked exceptions do not have that compiler requirement.
- The distinction is about checking, not whether an event occurs at runtime.

Example

```
Checked: IOException
Unchecked: NumberFormatException
Unchecked: IllegalArgumentException
```

Throwing and catching

- throw raises a particular exception object.
- throws appears in a method declaration.
- Place specific catches before broader compatible catches.

Example

```
if (marks < 0 || marks > 100) {
  throw new IllegalArgumentException(
      "marks outside 0..100");
}
```

Validation and parsing solve different problems

- Parsing checks whether text can represent a number of the requested kind.
- Validation checks whether the parsed number belongs to the application's allowed domain.
- The value 101 parses as int but violates a marks range of 0..100.

Example

```
"cat": NumberFormatException while parsing
"101": parses, then fails range validation
"100": passes both checks
```

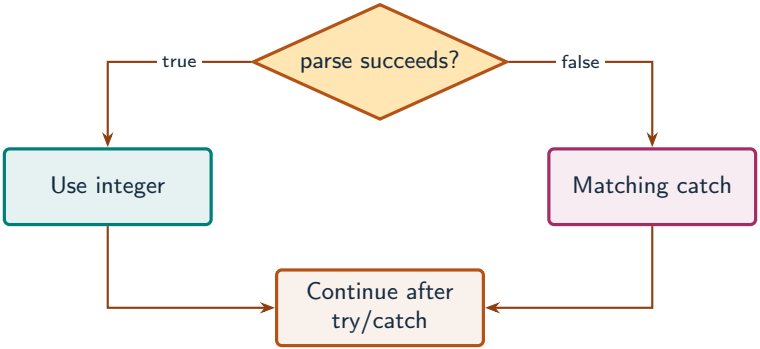
Exception handling changes the path

- throw immediately interrupts the ordinary path in the current block.
- A compatible catch can report the failure and allow later work to continue.
- A loop with a try-catch inside its body can examine later inputs after one failure.

Example

```
For inputs -1,0,100,101:
Reject the first, accept the next two, reject the last.
```

An exception transfers control to a handler



What to notice

When parsing fails, the remaining statements in that try block are skipped.

Key distinctions

Input	Parses as int?	Valid mark?	Outcome
"cat"	No	Not evaluated	Parsing failure
"-1"	Yes	No	Range failure
"0"	Yes	Yes	Accept
"100"	Yes	Yes	Accept
"101"	Yes	No	Range failure

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
try {
  int marks = Integer.parseInt("bad");
  System.out.println(marks);
} catch (NumberFormatException ex) {
  System.out.println("Invalid integer");
}
System.out.println("Finished");
```

First example: result

Invalid integer
Finished
The handler runs, then normal execution continues after the try-catch.

Checked and unchecked exceptions
Throwing and catching

Guided practice

Define validateMark(int marks) that throws IllegalArgumentException outside 0..100. Test -1, 0, 100 and 101.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Write a validator that throws when marks are outside 0..100.
- Call the validator for each boundary example inside a try block.
- Print a success message only after validation returns normally.

The next program uses fixed inputs so its result can be reproduced.

Complete solution (1/2)

```
public class Solution24 {
  public static void main(String[] args) throws Exception {
    int[] cases = {-1, 0, 100, 101};
    for (int mark : cases) {
      try {
        validateMark(mark);
        System.out.println(mark + " accepted");
      } catch (IllegalArgumentException ex) {
        System.out.println(mark + " rejected");
```

Complete solution (2/2)

```
      }
    }
  }
  static void validateMark(int mark) {
    if (mark < 0 || mark > 100) {
      throw new IllegalArgumentException("marks outside 0..100");
    }
  }
}
```

Execution trace

Mark	Invalid condition	Control path	Printed result
-1	true	throw then catch	-1 rejected
0	false	Normal return	0 accepted
100	false	Normal return	100 accepted
101	true	throw then catch	101 rejected

Follow the changing state in execution order.

Solution result

-1 rejected
0 accepted
100 accepted
101 rejected

Why this result follows
Print a success message only after validation returns normally.

A common incorrect approach

```
try { work(); }
catch (Exception ex) { }
catch (NumberFormatException ex) { }
```

Example

```
The specific handler is unreachable. Catch NumberFormatException first, and give each handler a
    meaningful action.
```

Boundary and failure checks

Check the stated input assumptions.
Read one integer and report malformed input using a specific catch. Keep parsing and range validation distinguishable.

Throw when marks<0 || marks>100. Inputs -1 and 101 fail. Inputs 0 and 100 return normally.

Check your understanding

Does throws itself create an exception? Must unchecked exceptions be declared?

Write a prediction and explain the rule that supports it.

Answer and explanation

No. throws declares a possible exception. Unchecked exceptions have no catch-or-declare compiler requirement.

The specific handler is unreachable. Catch NumberFormatException first, and give each handler a meaningful action.

Compare cases and predict outcomes

Token	Parsing outcome	Handler needed?
42	int 42	No
four	NumberFormatException	Yes
2147483648	Outside int range	Yes

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Parse "four" inside try/catch, print "Invalid integer" on failure, then print "Finished" after the handler. Predict both output lines.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
try {
  int n = Integer.parseInt("four");
  System.out.println(n);
} catch (NumberFormatException ex) {
  System.out.println("Invalid integer");
}
System.out.println("Finished");
```

- parseInt throws before n is printed.
- The matching handler prints Invalid integer.
- Normal control resumes after the try/catch and prints Finished.

Which exception is observed first?

- One statement can contain more than one potential failure. Execution order determines the observed exception.
- For a simple array assignment, the right-hand expression is evaluated before the final bounds check.
- Only the first thrown exception reaches a matching handler in this execution.

Source: supplied ISB campus slides. See speaker notes and ISB_Source_Map.md.

Worked problem: Which exception is observed first?

Task

Trace int[] a=new int[5]; a[5]=30/zero with zero=0, then compare the case zero=2.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture24 {
  public static void main(String[] args) throws Exception {
    int[] a = new int[5];
    int zero = 0;
    try {
      a[5] = 30 / zero;
    } catch (ArithmeticException ex) {
      System.out.println("Arithmetic exception");
    } catch (ArrayIndexOutOfBoundsException ex) {
      System.out.println("Array index exception");
    }
    System.out.println("Rest of the code");
  }

}
```

Worked trace and expected result

Divisor / index	First failure	Handler
0 / 5	Integer division by zero	ArithmeticException
2 / 5	Invalid index after RHS	ArrayIndexOutOfBounds Exception
2 / 4	None	Assignment succeeds

Expected console output

```
Arithmetic exception
Rest of the code
```

Extend the ISB example

Practice

What happens if catch(Exception ex) precedes both specific handlers?

Explain your reasoning

Compilation fails because the later subclass handlers are unreachable.

Lecture summary

- propagation and the exception hierarchy
- try, throw and catch
- checked and unchecked exceptions

- Write a validator that throws when marks are outside 0..100.
- Call the validator for each boundary example inside a try block.
- Print a success message only after validation returns normally.

After-class practice

Read one integer and report malformed input using a specific catch. Keep parsing and range validation distinguishable.

Syllabus reading
Deitel Ch 11
Next lecture: Cleanup and exception propagation